



A Taylor model toolbox for solving ODEs implemented in MATLAB/INTLAB

Florian Bünger

Institute for Reliable Computing, Hamburg University of Technology, Am Schwarzenberg-Campus 3, 21073 Hamburg, Germany



ARTICLE INFO

Article history:

Received 20 March 2019

Received in revised form 27 August 2019

MSC:

34-04

65L05

65G20

Keywords:

Taylor models

Initial value problems

Ordinary differential equations

ABSTRACT

The new INTLAB release V11 contains two verified ODE solvers. One is a MATLAB implementation of Lohner's classical AWA, the other one follows the so-called Taylor model approach which is the main subject of this article.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

Solving differential equations rigorously is a main and vigorous topic in the field of verified computation. Here, “solving rigorously” means that a computer program supplies an approximate solution along with error bounds that respect all numerical as well as all rounding errors that occurred during the computation. An exact solution is proved to be enclosed within these rigorous bounds.

Among the vast crowd of different kinds of differential equations the class of initial value problems (IVPs) for ordinary differential equations (ODEs) is the most basic one. Starting from Lohner's famous AWA [1] many other effective verified IVP solvers were developed using various different methods. For example, VNODE-LP [2] uses the Hermite–Obreschkoff method, COSY INFINITY [3] invented the Taylor model approach, and the kv library [4] utilizes a mixture of affine and verified power series arithmetic.

Other implementations using Taylor models, the main subject of this article, are VSPODE [5], FLOW [6], ODEintegrator [7], and RIOT [8]. However, COSY INFINITY developed by Berz and Makino and their group is the by far most advanced implementation of that approach. Sadly, due to some unfortunate, widely discussed patent issues, the source code of COSY INFINITY is not open which makes it difficult to retrace and compare results. That was one reason why we implemented an easy to use, easy to read, and easy to debug Taylor model toolbox in MATLAB/INTLAB [9] so that basic advantages and disadvantages of Taylor models may be better to assess.

The article is organized as follows. In Section 2 we shortly describe our MATLAB implementation of Lohner's classical AWA. In main Section 3 we start in Section 3.1 with an introduction to Taylor models, their rigorous arithmetic and the implementation of standard functions like exp, log, sin, etc. Afterwards, in Section 3.2, the Taylor model method for enclosing solutions of initial value problems

$$y' = f(t, y), \quad y(t_0) = y_0,$$

E-mail address: florian.buenger@tuhh.de.

where the initial value y_0 may be an interval vector, is outlined. Then, in Section 3.3, accompanying features like shrink wrapping, blunting, and preconditioning for fighting the wrapping effect and stabilizing long term integration are mentioned. Finally, in Section 3.4, results and run times for some test problems are given.

2. The AWA toolbox

Calling INTLAB-AWA is similar to calling MATLAB's nonverified ODE solvers like `ode45`:

```
[T, Y] = awa(odefun, jacobimat, tspan, y0)
[T, Y] = awa(odefun, jacobimat, tspan, y0, options)
[T, Y, Z] = awa(odefun, jacobimat, tspan, y0, options)
```

The first call, where $tspan = [t_0, t_f]$, integrates the system of differential equations $y' = f(t, y)$ from t_0 to t_f with initial conditions y_0 . Each row in the interval solution array Y corresponds to a value at time points returned in the floating-point column vector T . Precisely, this means $y_i(T(j))$ is contained in $Y(j, i)$ for $i = 1, \dots, n$ and $j = 1, \dots, m$ where n is the dimension of the ODE, $m := \text{length}(T)$, $T(1) = t_0$, and $T(m) = t_f$.

In other words: the j th row of Y is an inclusion of the n components y_i of the true solution y at time $t = T(j)$. The initial condition y_0 may be an interval vector. This means that the computed inclusion Y contains the true solution of the differential equation for any initial values $y_i(t_0)$ in the interval $y_0(i)$, $i = 1, \dots, n$. This allows to model uncertainties in the initial values.

The right-hand side $f(t, y)$ of the ODE and its Jacobian matrix w.r.t. y must be implemented by the user and passed to `awa` by corresponding function handles `odefun` and `jacobimat`, respectively. The optional input argument `options` is a structure with components serving as parameters to `awa`. The function `awaset` can be used to create options similar to MATLAB's function `odeset`. For example,

```
options = awaset('order', 20, 'h0', 1e-3, 'EvalMeth', 1);
```

sets the Taylor expansion order to 20, the initial step size h_0 to (approximately) 0.001, and chooses evaluation method 1: parallelepiped enclosure. Other evaluation methods are as in [1]:

0 : interval vector enclosure	3 : intersection of methods 0 and 1
2 : QR decomposition	4 : intersection of methods 0 and 2

Further options are the minimum step size $h_{\min}$ and the absolute and relative error tolerances `AbsTol` and `RelTol` which correspond to the parameters `E_A` and `E_R` of Lohner's original Pascal implementation. Finally, the optional output argument `Z` is a Taylor coefficient interval array which allows to compute enclosures at intermediate time points. Precisely, if t is such an intermediate time point in the interval (t_j, t_{j+1}) , then $y_i(t)$ is contained in the interval evaluation of

$$\sum_{k=0}^p Z(j, i, k) \cdot \left(\frac{t - t_j}{t_{j+1} - t_j} \right)^k, \quad p := \text{order}.$$

As an example we consider the van der Pol equation, a second order ODE

$$y'' - c(1 - y^2)y' + y = 0,$$

where $c > 0$ is a scalar parameter. Rewrite this equation as a system of first-order ODEs using the substitution $y_1 := y$ and $y_2 := y'$. The resulting system of first-order ODEs is

$$\begin{aligned} y_1' &= y_2, \\ y_2' &= c(1 - y_1^2)y_2 - y_1. \end{aligned}$$

The Jacobian matrix of the van der Pol system is

$$J = \begin{pmatrix} 0 & 1 \\ -2cy_1y_2 - 1 & c(1 - y_1^2) \end{pmatrix}.$$

The following functions `vdp_fun` and `vdp_jac` implement the van der Pol equation and its Jacobian matrix for $c := 1$.

```
function dydt = vdp_fun(t,y)
dydt = [ y(2) ; (1-sqr(y(1))).*y(2)-y(1) ];

function J = vdp_jac(t,y)
J = typeadjust([0,1;0,0],y);
J(2,1) = -2.*y(1).*y(2) - 1;
J(2,2) = 1 - sqr(y(1));
```

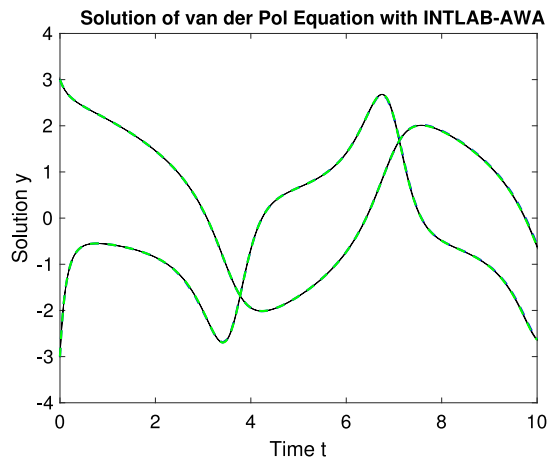


Fig. 1. van der Pol equation.

The ODE shall be solved on the time interval $[0, 10]$ with interval initial values $Y_0 = [2.999, 3.001] \times [-3.001, -2.999]$. The resulting output is a floating-point column vector of time points T and a solution interval array Y of data type interval. Each row in Y corresponds to a time returned in the corresponding row of T . The first column of Y contains inclusions for y_1 , and the second column for y_2 . In MATLAB we execute

```
[T,Y] = awa(@vdp_fun,@vdp_jac,[0,10],midrad([3;-3],1e-3));
awa_disp(T,Y); % formatted result display
```

and obtain the following enclosures and their diameters at the final time point $t_f = 10$:

```
...
t = 10
[y_1] = [ -0.6302, -0.5973]    d([y_1]) = 3.28e-2
[y_2] = [ -2.6416, -2.6271]    d([y_2]) = 1.45e-2
```

The enclosures for y_1 and y_2 against t can be plotted by using INTLAB's plot function. For comparison, the system is also solved twice with MATLAB's nonverified ODE solver ode45 with initial condition $y_0 = [3, -3]$; the midpoint of Y_0 . First, we use the default options for ode45 and store the result in $[t, v]$. Second, we choose ode45 options for higher accuracy and store the result in $[s, w]$. The inclusion computed by awa and both approximations by ode45 are displayed in one plot by the following instructions (see Fig. 1):

```
plot(T,Y); % Plot awa enclosures for y_1 and y_2.
title('Solution of van der Pol Equation with INTLAB-AWA');
xlabel('Time t'); ylabel('Solution y'); hold on;

% Execute ode45 with default options.
[t,v] = ode45(@vdp_fun,[0,10],[3;-3]);

% Execute ode45 with higher accuracy.
options = odeset('RelTol',1e-12,'AbsTol',[1e-12;1e-12]);
[s,w] = ode45(@vdp_fun,[0,10],[3;-3],options);

plot(t,v,'--b',s,w,'--g'); % Plot both ode45 results.
```

Zooming in on the plot for the first component y_1 at $t = 2$ shows that both ode45 results, v_1 (blue dashed line) and w_1 (green dashed line), are contained in the enclosure $Y_1 := Y(:, 1)$ computed by awa (red tube). Zooming in on the plot at the final time $t = 10$ shows that the numerical approximation v_1 is no longer contained in the awa enclosure, thus cannot be correct. Contrary, w_1 is still quite centered in the red tube confirming its higher accuracy (see Fig. 2).

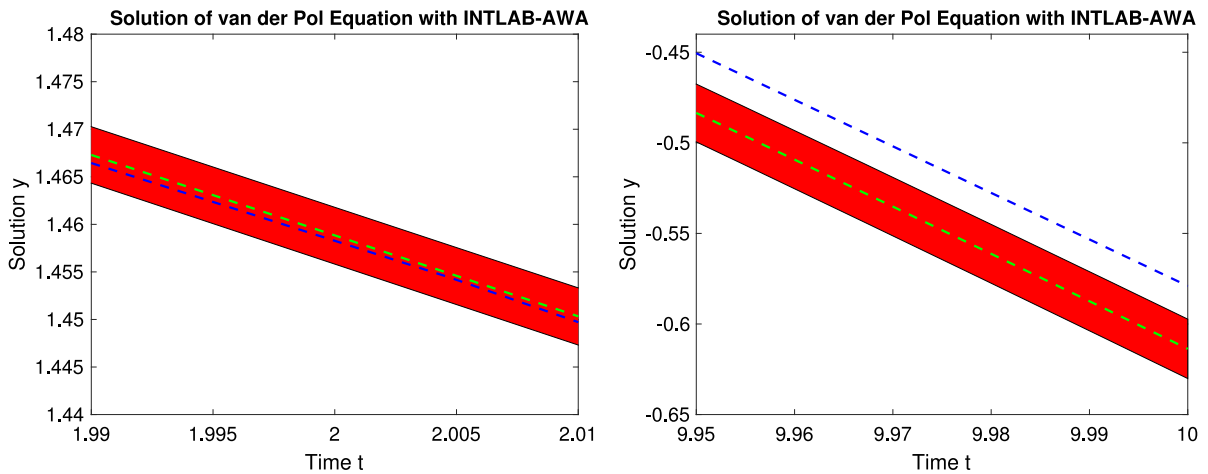


Fig. 2. van der Pol equation zoomed.

3. The Taylor model toolbox

3.1. Definition and arithmetic of Taylor models

The simplest way to think of a Taylor model is to regard it as a multivariate, real polynomial p of bounded degree m in a fixed number of k variables. Such a polynomial is usually written as

$$p(x) = \sum_{\alpha, |\alpha| \leq m} p_{\alpha} x^{\alpha}, \quad |\alpha| := \alpha_1 + \dots + \alpha_k, \quad x^{\alpha} := x_1^{\alpha_1} \dots x_k^{\alpha_k},$$

where $\alpha = (\alpha_1, \dots, \alpha_k)$ is a multiindex consisting of nonnegative integer exponents α_i , and the polynomial coefficients p_{α} are real numbers. Now, a k -dimensional domain

$$D = [u_1, v_1] \times \dots \times [u_k, v_k]$$

is fixed which is simply an interval vector¹ of length k with interval components $[u_i, v_i]$, $i = 1, \dots, k$. Next, a “center point” $c = (c_1, \dots, c_k)$ in D is fixed which must not necessarily be the exact center $(u + v)/2$. For example, $c := u$ or $c := v$ is also allowed, only c_i must be in $[u_i, v_i]$ for $i = 1, \dots, k$. The standard domain and center point are

$$D^s := [-1, 1] \times \dots \times [-1, 1], \quad c^s := (0, \dots, 0).$$

The centered image

$$p(D - c) = \{p(x - c) \mid x \in D\}$$

is the enclosure represented by the data p , D , and c . For the special case of degree $m = 1$, standard domain D^s , and center point c^s this coincides with the enclosure represented by an object in affine arithmetic with k error terms.

Moreover, a Taylor model contains a remainder/error interval E which absorbs the inevitable rounding and degree truncation errors. Thus, the enclosure (range) represented by the Taylor model data p , D , c , E enlarges to

$$p(D - c) + E = \{p(x - c) + e \mid x \in D, e \in E\}. \quad (1)$$

We remark that in our implementation the polynomial coefficients p_{α} , the domain bounds u_i , v_i , and the center points c_i are restricted to be floating-point numbers in order to keep them representable on a computer. All rounding errors that go along with this passing from real to floating-point numbers are absorbed in the remainder interval E . Although this is a crucial, tedious programming task, it is somehow standard in the area of verified computation and will therefore be dropped in what follows.

For a single Taylor model $y := (p, D, c, E)$ the range is a closed interval. In particular, it is convex. Taking a second Taylor model $z := (q, D, c, F)$ with same domain and center point, the range R of the Taylor model vector $w := (y, z)$ is

¹ As usual, the set of real interval vectors of length k will be denoted by $\mathbb{IR}^k$, and, for $k = 1$, $\mathbb{IR}$ is the set of closed real intervals.

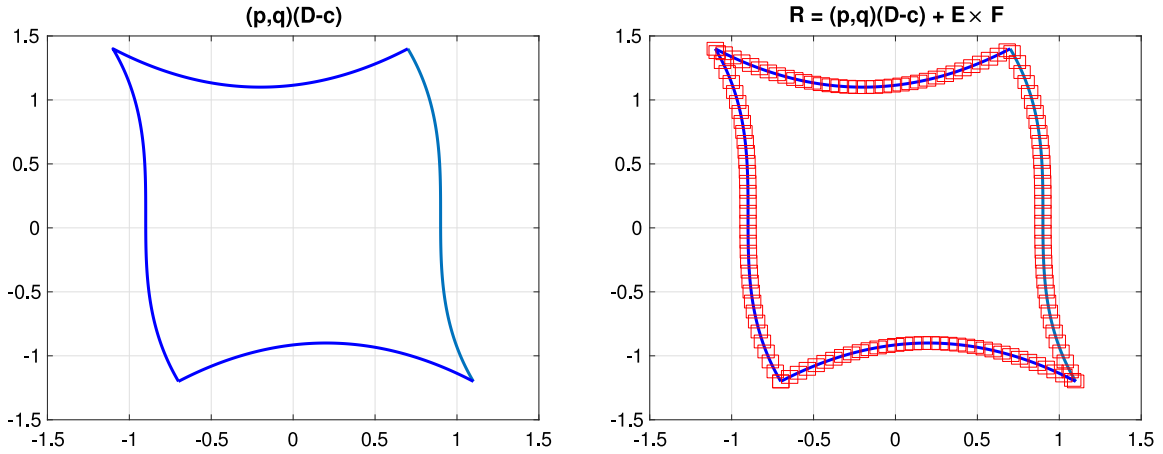


Fig. 3. Taylor model range.

defined by

$$\begin{aligned} R &:= \{(p(x-c) + e, q(x-c) + f) \mid x \in D, e \in E, f \in F\} \\ &= (p, q)(D-c) + E \times F. \end{aligned} \quad (2)$$

This set R is not necessarily convex anymore. This is the big advantage of Taylor models: they allow to enclose higher dimensional shapes with curved boundaries without too much overestimation. For example, take $k := 2$, $m := 3$, $D = [-1, 1]^2$, $c := (0, 0)$, $E = F := [-0.05, 0.05]$, $p(x_1, x_2) := -0.2x_2^3 + 0.9x_1$, and $q(x_1, x_2) := 0.3x_1^2x_2 + x_2 + 0.1$. The left picture of Fig. 3 shows the polynomial image

$$S := (p, q)(D-c) = \{(p(x_1, x_2), q(x_1, x_2)) \mid x_1, x_2 \in [-1, 1]\}$$

bounded by the blue line while the right picture shows the whole Taylor model range R which is S rendered by the remainder square $E \times F = [-0.05, 0.05]^2$ plotted in red, cf. (2). In practice, the rendering remainder boxes are small compared to the leading sets S so that they do not destroy their curved boundary shapes. This is important because the latter carry and evolve the main information on the considered problem.

From a formal mathematical point of view, a Taylor model given by the data p, D, c, E, m, k as above is defined as²

$$p + E := \{f \in C(D, \mathbb{R}) \mid \forall x \in D : f(x) \in p(x-c) + E\}. \quad (3)$$

Thus, a Taylor model $p + E$ is a set of real-valued, continuous functions with domain D that at each point $x \in D$ deviate from $p(x-c)$ by an element $e \in E$. The plus sign in $p + E$ has only a suggestive meaning, it does not implicitly define some obscure summation of the polynomial part p and the remainder interval part E .

The idea behind definition (3) is to consider p as the Taylor polynomial of order m with evaluation point c of an $(m+1)$ -times differentiable function $f \in C^{m+1}(D, \mathbb{R})$ such that the remainder term $r(x) := f(x) - p(x-c)$ is enclosed in E for all $x \in D$. In that light $p + E$ is also called an m th order Taylor model for $f \in C^{m+1}(D, \mathbb{R})$. This point of view explains the name “Taylor model”, although definition (3) on its own does not necessitate any Taylor expansion of some unknown function f in the background.

Taylor model vectors are analogously denoted by $p + E$ where now

$$p = (p_1, \dots, p_n) \in (\mathbb{R}[x_1, \dots, x_k])^n$$

is a vector of n multivariate polynomials p_i and $E = E_1 \times \dots \times E_k$ is an interval vector such that each $p_i + E_i$ is a Taylor model as defined in (3) for $i = 1, \dots, n$.

Next, arithmetic operations $+$, $-$, $\times$, $\div$, standard functions like $\exp$, $\log$, $\sin$, $\cos$, etc., and also integration with respect to one of the k variables can be defined on the set of all m th order Taylor models with common, fixed D, c .

Summation and subtraction are easily defined by $(p + E) \pm (q + F) := (p \pm q) + (E \pm F)$ where $(E \pm F)$ is done in interval arithmetic. As stated before, rounding errors that occur in the polynomial summation/subtraction $p \pm q$ are transferred to the remainder interval $K := E \pm F$ without further mentioning for ease of presentation.

For the multiplication of $p + E$ and $q + F$ write $pq = r + s$ such that $\deg(r) \leq m$, where $\deg(r)$ denotes the degree of r , and such that all terms of s are of order greater than m . Then, formally,

$$(p + E)(q + F) = pq + pF + E(q + F) = r + (s + pF + E(q + F)) = pq + qE + F(p + E) = r + (s + qE + F(p + E)).$$

² $C(D, \mathbb{R})$ denotes the set of all continuous functions $f : D \rightarrow \mathbb{R}$. In general $C^k(D, \mathbb{R})$, $k \in \mathbb{N}_0$, denotes the set of all k -times continuously differentiable functions $f : D \rightarrow \mathbb{R}$.

Now, use interval arithmetic to compute

$$G_1 := s(D - x_0) + p(D - x_0)F + E(q(D - x_0) + F),$$

$$G_2 := s(D - x_0) + q(D - x_0)E + F(p(D - x_0) + E),$$

and define $G := G_1 \cap G_2$ and $(p + E) \cdot (q + F) := r + G$.

For a standard function f like exp or log the Taylor model evaluation of $f(p + E) := r + G$ is done as follows. For simplicity we assume $0 \in E$ which, in practice, is (almost) always the case. Interval arithmetic is used to enclose the range $p(D - c) + E$ of $p + E$, cf. (1), in an interval G_1 . Since $c \in D$ and $0 \in E$ we have $z := p(0) \in p(D - c) + E \subseteq G_1$. Now, G_1 is thought to be centered around z and Taylor expansion of f at $x \in G_1$ around z becomes

$$f(x) = \underbrace{\sum_{i=0}^m \frac{f^{(i)}(z)}{i!} (x - z)^i}_{q(x):=} + \underbrace{\frac{f^{m+1}(\xi)}{(m+1)!} (x - z)^{m+1}}_{\text{rest}(x):=}$$

where ξ is an intermediate value on the line between z and x which is contained in G_1 because G_1 is convex. The term $q(p + E) = r + F$ is computed in Taylor model arithmetic by iterative summation and multiplication as defined before. For example, Horner's scheme can be used for that.³ Then, formally,

$$f(p + E) = q(p + E) + \text{rest}(p + E) = r + (F + \text{rest}(p + E)).$$

The range of the Taylor model $\text{rest}(p + E)$ is enclosed by ordinary interval arithmetic

$$\text{rest}(G_1) \subseteq \frac{f^{m+1}(G_1)}{(m+1)!} (G_1 - z)^{(m+1)} =: G_2.$$

Taking $G := F + G_2$ we define $f(p + E) := r + G$.

Integrating a Taylor model $p + E$ with respect to one variable $x_i \in [u_i, v_i]$, $1 \leq i \leq k$, is comparatively easy. First, integrate p with respect to x_i . The resulting polynomial $\hat{p}$ has degree at most $m + 1$. Then, split $\hat{p} = r + s$ such that s contains all terms of degree $m + 1$ and

$$\int_{c_i}^{x_i} p(x - c) + E \, dx_i \subseteq r(x - c) + \underbrace{\{s(D - c) + E \cdot ([u_i, v_i] - c_i)\}}_F.$$

The trailing part in curly braces is evaluated in interval arithmetic and enclosed in F . The antiderivative of $p + E$ with respect to x_i is then defined by

$$\int_{c_i}^{x_i} p + E \, dx_i := r + F.$$

As stated before, the main advantage of Taylor models is that they allow to enclose multidimensional shapes with curved, not necessarily convex boundaries without too much overestimation. This is not possible for traditional interval arithmetic which only supplies enclosures in axis parallel boxes. It is also not possible for affine arithmetic which only provides convex polytopes as enclosure sets. As an example consider the square box $B := [-0.1, 0.1] \times [-0.1, 0.1]$ which shall be mapped iteratively by the function

$$f(x, y) := (x - y(0.125 + 2y), y + 6x^3).$$

The following program executes four such iterations with interval arithmetic (blue plot), affine arithmetic (green plot), and Taylor model arithmetic (black plot). For each iteration the curved red line shows the approximate true boundary of the transformed box B , i.e., the boundary of the sets $f(B)$, $f(f(B))$, $f(f(f(B)))$, and $f(f(f(f(B))))$, respectively. The four pictures clearly show that Taylor model arithmetic creates almost no overestimation compared to the rough inclusions supplied by interval arithmetic and affine arithmetic (see Fig. 4).

```
f = @(x,y) [ x - y.*(0.125 + 2*y) , y + 6*x.^3 ]; % function f
r = 0.1; B = midrad([0;0],r); % box B
a = (-1:0.01:1)';
e = ones(length(a),1);
b = r*[a e; e -a; -a -e; -e a];
D.inf = [-1;-1]; % standard domain D = [-1,1]x[-1,1]
D.sup = [1;1];
c = [0;0]; % standard center point c = (0,0)
```

³ Since Taylor model summation and multiplication are heavily used in the implementation of each standard function, these two arithmetic operations, and especially multiplication, are the overall performance bottleneck and must be implemented as effective as possible!

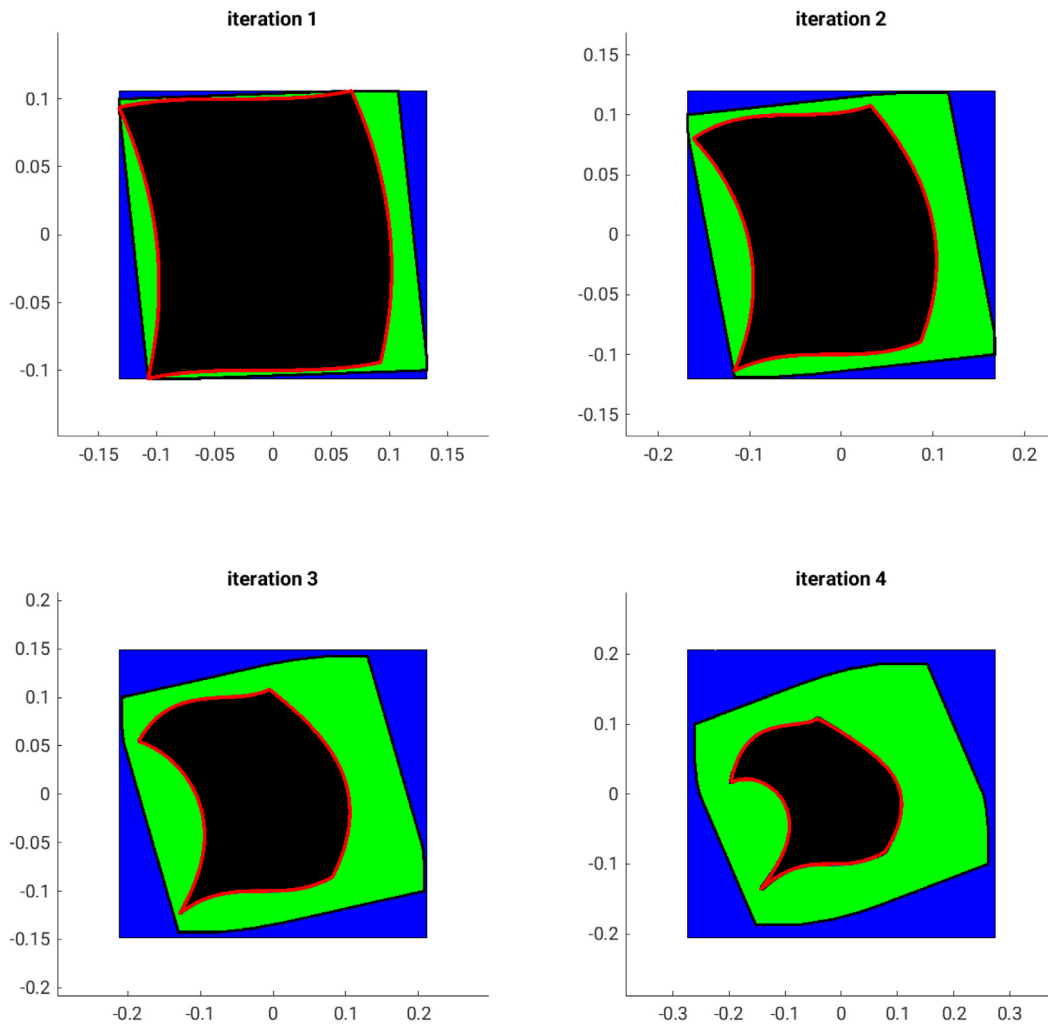


Fig. 4. Taylor model arithmetic vs. interval and affine arithmetic.

```

y_fl = b;          % floating-point    version of B
y_iv = B;          % interval          version of B
y_af = affari(B);  % affine arithmetic version of B
                  % Taylor model      version of B (order := 18)
y_tm = taylormodelinit({c;c},{D;D},[18;18],[[1 0];[0 1]},{r;r});

for i = 1:4
    subplot(2,2,i); title(['iteration ', num2str(i)]); hold on
    y_iv = f(y_iv(1),y_iv(2));          % Apply f in interval arithmetic
    plotintval(y_iv','b');              % ... and plot enclosure in blue.
    y_af = f(y_af(1),y_af(2));          % Apply f in affine arithmetic
    plotaffari(y_af','g');              % ... and plot enclosure in green.
    y_tm = f(y_tm(1),y_tm(2));          % Apply f in Taylor model arithmetic
    plottaylormodel(y_tm,[],[],100)    % ... and plot enclosure in black.
    y_fl = f(y_fl(:,1),y_fl(:,2));      % Compute boundary of f(f(...(B)))
    plot(y_fl(:,1),y_fl(:,2),'r');      % ... and plot it as red line.
end

```

3.2. Solving IVPs with interval initial values

Let an interval initial value problem for an n -dimensional ODE be given:

$$y' = f(t, y) \in \mathbb{R}^n, \quad t \in [t_0, t_f] \quad (4)$$

$$y(t_0) \in Y_0 \in \mathbb{IR}^n. \quad (5)$$

The problem is solved stepwise on time subintervals $[t_i, t_{i+1}]$ where the time grid $t_0 < t_1 < \dots < t_N = t_f$ is chosen by some automatic step size control which will not be explained in greater detail. For each $[t_i, t_{i+1}]$ a Taylor model vector

$$p^{(i)}(x, t) + E^{(i)} \quad (6)$$

of length n with polynomial part $p^{(i)}(x, t) \in \mathbb{R}[x, t]^n$ and remainder interval vector $E^{(i)} \in \mathbb{IR}^n$ is computed such that

$$\forall t \in [t_i, t_{i+1}] : y(t) \in p^{(i)}([-1, 1]^n, t - t_i) + E^{(i)}. \quad (7)$$

In words: at each time point $t \in [t_i, t_{i+1}]$ the function value $y(t)$ of any solution y of (4) with initial value $y(t_0) \in Y_0$ is contained in the set on the right-hand side of (7) which is the range of the t -fixed Taylor model $p^{(i)}(x, t - t_i) + E^{(i)}$ on the standard domain $B := D^s = [-1, 1]^n$ with center point zero.

Using the notation from the previous subsection, the ODE Taylor model $p^{(i)}(x, t) + E^{(i)}$ has dimension $k := n + 1$ where n is the dimension of the ODE and the additional dimension is for the time t . The Taylor model domain and center point are $D := B \times [t_i, t_{i+1}]$ and $c := (0, \dots, 0, t_i) \in \mathbb{R}^{n+1}$, respectively. Thus, the space-like variables $x_1, \dots, x_n$ have standard domain $[-1, 1]$ and standard center point zero, and the time variable t has domain $[t_i, t_{i+1}]$ with “center point” t_i which actually is the left boundary point of the time domain.

Without going into details, the main algorithm steps for computing the ODE Taylor models (6) are as follows.

Step (1) Write $Y_0 = y_0 + [-r, r]$ in midpoint radius notation with midpoint $y_0 \in \mathbb{R}^n$ and radius vector $r \in \mathbb{R}_{\geq 0}^n$. Then, the Taylor model $q(x) + J$ with polynomial part $q(x) := y_0 + \text{diag}(r)x$, $x \in B$, and zero remainder $J := [0, 0] \in \mathbb{IR}^n$ is independent from t and has the range $\hat{Y}_0 := q(B) + J = Y_0$. Thus, it parameterizes the initial value set Y_0 .

Step (2) A solution y of (4) on $[t_0, t_1]$ with point initial value $\hat{y}_0 := y(t_0) \in \hat{Y}_0$ is a fixed-point of the integral operator

$$K : C([t_0, t_1], \mathbb{R}^n) \rightarrow C([t_0, t_1], \mathbb{R}^n), \quad z \mapsto \hat{y}_0 + \int_{t_0}^t f(\tau, z(\tau)) d\tau.$$

This integral operator is generalized to ODE Taylor models $p(x, t) + E$ as defined above instead of continuous maps z :

$$K(p + E) := (q + J) + \int_{t_0}^t f(\tau, p + E) d\tau. \quad (8)$$

The right-hand side is evaluated in Taylor model arithmetic as defined before. That requires the ODE function f to be a concatenation of standard functions and arithmetic expressions, and this is always tacitly assumed.

Step (3) The crucial task is to find an ODE Taylor model $p + E$ such that

$$K(p + E) \subseteq p + E. \quad (9)$$

Recall from (3) that Taylor models are sets of continuous functions and inclusion (9) is meant that way. If it holds true, then, using Schauder's fixed-point theorem, for example, it can be deduced that all solutions y of (4), (5) fulfill

$$y(t) \in p(B, t - t_0) + E \quad \forall t \in [t_0, t_1].$$

This is (7) for $i = 0$, $p^{(0)} := p$, and $E^{(0)} := E$.

Step (4) For continuing the integration on the next time interval $[t_1, t_2]$, set $q(x) := p(x, t_1 - t_0)$, $J := E$, $\hat{Y}_0 := q(B) + J$, and repeat steps (2), (3) with replacing t_0, t_1 by t_1, t_2 to compute $p^{(1)} + E^{(1)}$ satisfying (7) for $i = 1$ and so forth.

Note that, after the first step, $\hat{Y}_0 := q(B) + J$, which is the initial value set for the next integration step, is in general not an interval vector like Y_0 anymore. It is a Taylor model range that may have curved boundaries representing the ODE flow at $t = t_1$. Note also that this set is only implicitly given by the Taylor model $q + J$ and that this suffices to define the integral operator (8).

The main inclusion step 3 is decomposed into two substeps, a Picard iteration followed by some ε -inflation. In a different form these standard steps also appear in other verified ODE solvers like AWA [1] or kv [4].

Step (3.1) Carrying on the notation of step 3, the Picard iteration in Taylor model form reads

$$p_{(0)} + I_{(0)} := q + J, \quad p_{(i+1)} + I_{(i+1)} := K(p_{(i)} + I_{(i)}). \quad (10)$$

It is well-known that m such recursive applications of the integral operator K reproduce the m th order Taylor polynomial of the solution of the IVP. Thus, for fixed Taylor model order m , the polynomial $p := p_{(m)}$ is taken as a candidate for (9) and we are left with determining an appropriate remainder E which will be done by ε -inflation in the next step. This points to an important performance issue for the implementation of the Picard iteration: it can be carried out in nonverified Taylor model arithmetic since only the polynomial part p and not the remainder term E is derived. Nonverified Taylor

model arithmetic means that only polynomial parts of Taylor models and no remainder terms are computed. We remark that instead of a Picard iteration the Lie derivative can be used to compute the polynomial part p , see [10], p. 80, sec. 2.2, and [6], p. 62, sec. 3.3.1. This is not implemented in `verifyode`.

Step (3.2) The ε -inflation for determining the remainder term E of (9) proceeds as follows⁴:

- (I) Start with the zero interval vector $E_{(0)} := [0, 0] \in \mathbb{R}^n$.
- (II) Evaluate the integral operator $p + E'_{(i)} := K(p + E_{(i)})$ in verified Taylor model arithmetic.
- (III) If $E'_{(i)} \subseteq E_{(i)}$, then $E := E'_{(i)}$ fulfills (9) and we are done. Otherwise, perform an inflation of $E'_{(i)}$ by

$$E_{(i+1)} := [1 - \varepsilon, 1 + \varepsilon] \cdot E'_{(i)} + [-\delta, \delta], \quad \varepsilon, \delta > 0 \quad (11)$$

and repeat (II).

If this iteration does not stop after a few loops, then the time step size may be decreased and the whole process for that specific time step must be repeated.

Evaluating the integral operator K , which more or less means evaluating the ODE function f , is the key performance task, especially in verified mode. To reduce these costs we implemented in addition to INTLAB's object-oriented interval arithmetic a simple non-object-oriented interval arithmetic since MATLAB's object-orientation is very slow. This causes a speed up factor of up to three compared to INTLAB's standard interval arithmetic.

Even more time-saving is the so-called record feature proposed by Eble [8], p. 112, for computing the remainder term E in step 3.2. Also FLOW implements such a feature, see [6], p. 73, sec. 3.3.3 "Fast remainder refinement", where Example 3.3.10 on p. 74 thoroughly describes this technique. We strongly recommend to take up this clever idea although, like nonverified Taylor model arithmetic, it must be implemented in all arithmetic operations and standard functions. Without going into details, the idea of the record feature is to compute polynomial parts of Taylor models in step (II) only once for $i = 0$ and write all their influences to remainder terms that occur during the whole evaluation of K to memory. Since these polynomial parts do not change for the subsequent loops $i = 1, 2, \dots$, these influencing quantities can be read from memory and do not have to be recomputed again. At first glance this may look very tedious and intransparent, but in fact it is not, and it is worth it.⁵

The following simplified univariate and time-independent example may give an impression of how the record feature works. Let $a(x) := x$ and $b(x) := (1 + 2\mathbf{u})x$, where $\mathbf{u}$ denotes the relative rounding error of some floating-point format, and consider summation of two Taylor models $a + I_a$ and $b + I_b$ having error intervals I_a and I_b at some intermediate step of the evaluation of the integral operator K in (II). Since $2 + 2\mathbf{u}$ is not a floating-point number, the coefficient of the sum $c(x) := a(x) + b(x) = (2 + 2\mathbf{u})x$ is not representable on a computer so that rounding is necessary. For example, we may take $\hat{c}(x) := 2x$ as a representable approximate. The range of the error $r(x) := c(x) - \hat{c}(x) = 2\mathbf{u}x$ is enclosed by the interval $J := r([-1, 1]) = [-2\mathbf{u}, 2\mathbf{u}]$. This only depends on the polynomial parts $a(x)$ and $b(x)$ and not on the error intervals I_a and I_b . The computed Taylor model sum is $\hat{c} + I_{\hat{c}}$ where $I_{\hat{c}} := I_a + I_b + J$ is evaluated in interval arithmetic. Since the polynomial parts $a(x)$ and $b(x)$ do not change during ε -inflation from one iteration to the next, also their influence $J = r([-1, 1])$ on $I_{\hat{c}}$ does not change. Therefore, it suffices to compute J only once in the first iteration and to read it from memory in all subsequent iterations in which I_a and I_b may assume different values. Furthermore, $\hat{c}(x)$ does not have to be stored or recomputed at all, since only the resulting error term $I_{\hat{c}}$ is of interest.

3.3. Fighting the wrapping effect

It is well-known that the Taylor model method described so far is not sufficient. It still suffers from the well-known wrapping effect and may therefore be called "naive" Taylor model method, cf. [12]. To overcome the wrapping effect Berz and Makino [13–15] invented several accompanying methods, e.g.: shrink wrapping, blunting, preconditioning, error parametrization, and dynamic domain decomposition. These methods are applied at the beginning of an integration step to the time independent Taylor model $q + J$ of step 4 of the previous subsection.

Shrink wrapping perturbs the polynomial part q to $\tilde{q}$ such that $q(B) + J \subseteq \tilde{q}(B) + \tilde{J}$ for a new remainder $\tilde{J}$ which is almost zero. It may look contradictory that enlarging the range can finally produce tighter enclosures. But, since $\tilde{J}$ is much smaller than J , inner dependencies of subsequent computations are better resolved by $\tilde{q} + \tilde{J}$. For details see [16], where we proposed a different version of shrink wrapping compared to [13]. We remark that although shrink wrapping gives good results in several cases, it is in general less effective than preconditioning or error parametrization.

Blunting simply regularizes the linear part of a Taylor model vector and can be combined with shrink wrapping and parallelepiped preconditioning. In principle, such a regularization was already suggested by Lohner [1], p. 51, paragraph 2, but he did not implement it in AWA. For details on blunting we refer also to [17].

Preconditioning factorizes $q + J$ into left and right Taylor models $q_\ell + J_\ell$ and $q_r + J_r$ such that

$$\forall x \in B: \quad q(x) + J \subseteq [(q_\ell + J_\ell) \circ (q_r + J_r)](x).$$

⁴ The epsilon-inflation was introduced and analyzed in [11]. Note that in particular the absolute perturbation is important.

⁵ To my knowledge COSY does not use such a feature.

Here, $\circ$ means concatenation of Taylor models which is straightforwardly defined based on multiplication and summation by $(q_\ell + J_\ell) \circ (q_r + J_r) := q_\ell(q_r + J_r) + J_\ell$. Moreover, the left remainder J_ℓ is aimed to be almost zero and $q_r(B) + J_r \subseteq B$ must hold true. For the first integration step, $q_r(x) := x$ is the identity and J_r is zero. Integrating the left Taylor model $q_\ell + J_\ell$ on $[t_i, t_{i+1}]$ yields a Taylor model $p_\ell^*(x, t) + I_\ell^*$ such that

$$(p_\ell^* + I_\ell^*) \circ (q_r + J_r)$$

factorizes the ODE-flow on $[t_i, t_{i+1}]$. For the next integration step write

$$p_\ell^*(x) := p_\ell^*(x, t_{i+1} - t_i) = Ax + b(x) + c, \quad x \in B \quad (12)$$

where the real $n \times n$ -matrix A represents the linear, $b(x) \in \mathbb{R}[x]^n$ the nonlinear, and $c = p_\ell^*(0) \in \mathbb{R}^n$ the constant part of $p_\ell^*(x)$. Preconditioning targets the linear part A which is factorized as $A = QR$. For the trivial factorization $Q := A$ and R the identity matrix this is the so-called parallelepiped method. In general more stable is a permuted QR factorization $AP = QR$, $R := \tilde{R}P^T$ with orthogonal Q , triangular $\tilde{R}$, and permutation matrix P which sorts the columns of A by Euclidean length in descending order as proposed by Lohner [1].⁶ For the chosen factorization compute in Taylor model arithmetic

$$\begin{aligned} u + F &:= Q^{-1}((p_\ell^* - c + I_\ell^*) \circ (q_r + J_r)) \\ &= (Rx + Q^{-1}b(x)) \circ (q_r + J_r) + Q^{-1}I_\ell^* \end{aligned}$$

and choose a diagonal scaling matrix S such that $S(u(B) + F) \subseteq B$. The new left and right Taylor models for the next integration step are then defined as

$$\begin{aligned} q_{r, \text{new}} + J_{r, \text{new}} &:= S(u + F) \\ q_{\ell, \text{new}} + J_{\ell, \text{new}} &:= QS^{-1}x + c \end{aligned}$$

where the left remainder term $J_{\ell, \text{new}}$ only covers rounding errors and is therefore again almost zero. For more details on preconditioning we also refer to [12].

Further techniques like error parametrization, which introduces additional variables to parametrize the remainder J , and automatic domain decomposition are described in [15]. Especially error parametrization seems to be very effective. In a different way such a parametrization is also done by Dzetkulič [7].

Until now, our MATLAB implementation only supports shrink wrapping, blunting, and preconditioning. For the later, the parallelepiped and the QR factorization method, as described above, are available; curvilinear preconditioning [14] is not implemented.

3.4. Numerical examples

In this section we consider several examples. Computations for our Taylor model based ODE solver `verifyode` were done under Linux openSUSE Leap 15.0 on a desktop PC having 15.6 GiB RAM and 8 x Intel Xeon E5-1620 0 @ 3.60 GHz CPU. Results for COSY and other verified solvers are solely taken from literature.⁷ Therefore, computation times cannot be compared directly. Also computed enclosures must be considered with caution since newer versions of the respective solvers may perform differently. Nevertheless, the stated results may at least give an impression.

In what follows we will investigate:

1. specific linear ODEs
2. the Lorenz system
3. the Lotka–Volterra equations
4. long term integration of a Kepler problem for asteroid motion
5. the double pendulum

First, we consider three linear examples $y' = By$ stated in [12], sec. 7:

$$\begin{aligned} (1) \ B &:= \begin{bmatrix} -0.4375 & 0.0625 & -0.2651650429 \\ 0.0625 & -0.4375 & -0.2651650429 \\ -0.2651650429 & -0.2651650429 & -0.375 \end{bmatrix} \sim \begin{bmatrix} -\frac{1}{2} & 0 & 0 \\ 0 & -\frac{3}{4} & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ (2) \ B &:= \begin{bmatrix} 0 & -0.7071067810 & -0.5 \\ 0.7071067810 & 0 & 0.5 \\ 0.5 & -0.5 & 0 \end{bmatrix} \sim \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ (3) \ B &:= \begin{bmatrix} -0.125 & -0.8321067810 & -0.3232233048 \\ 0.5821067810 & -0.125 & 0.6767766952 \\ 0.6767766952 & -0.3232233048 & -0.25 \end{bmatrix} \sim \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & -\frac{1}{2} \end{bmatrix} \end{aligned}$$

⁶ Other factorizations like curvilinear coordinates [14] may be chosen according to the considered problem.

⁷ For copyright reasons COSY is not available to us. Thus, we use older published data.

The matrices B are, up to minor perturbations, conjugates of the matrices on the right-hand side. Thus, the first is a contraction with eigenvalues $-1/2, -3/4, 0$. The second has eigenvalues $\pm i, 0$ and is therefore a rotation of a hyperplane. The third has eigenvalues $\pm i$ and $-1/2$ and is a mixture of a rotation and a contraction.

Integration is done for interval initial values $y_0 := (1, 1, 1)^T \pm 0.001 \in \mathbb{R}^3$ for the time period $t \in [0, 100]$. In [12], COSY was executed with minimum step size $h_{\min} := 0.25$, Taylor model order $m := 12$, and with shrink wrapping (sw) as well as with QR preconditioning (QR). We also used these settings for our solver `verifyode`. The following table compares the obtained results.⁸ The third column shows the number of integration steps of the specific method, and the fourth column displays the interval enclosure of the first component y_1 at end of integration $t = 100$. The final column shows the computing time, but since COSY and `verifyode` were executed on different computers they are not well comparable. Because COSY is compiled Fortran code, whereas `verifyode` is interpreted MATLAB code, run time comparison is not really fair anyway.

Case		Steps	$y_1(100)$	Time [s]
(1)	COSY sw	272	$[-2.282e+112, 2.282e+112]$	5.92
	<code>verifyode</code> sw	–	$h < h_{\min} \Rightarrow \text{stop}$	–
	COSY QR	122	$1.4 \frac{73001618611069}{55930550902344} e-001$	4.67
	<code>verifyode</code> QR	113	$1.4 \frac{73001618608807}{55930550904611} e-001$	14.06
(2)	COSY sw	343	$1.49 \frac{5212933099568}{2225494495548} e+000$	5.92
	<code>verifyode</code> sw	279	$1.49 \frac{5784411031748}{1654016563373} e+000$	28.94
	COSY QR	343	$1.49 \frac{5212933089804}{2225494504794} e+000$	8.28
	<code>verifyode</code> QR	280	$1.49 \frac{5212933012279}{2225494582769} e+000$	34.63
(3)	COSY sw	357	$[-3.566e+106, 3.566e+106]$	7.18
	<code>verifyode</code> sw	293	$1.34 \frac{9660689604455}{4884500576376} e+000$	27.79
	COSY QR	362	$1.34 \frac{8619867744207}{5925322436435} e+000$	9.05
	<code>verifyode</code> QR	283	$1.34 \frac{8619867686266}{5925322494503} e+000$	32.47

For the first test case COSY executed with shrink wrapping needs 272 integration steps and finally gets a wide, meaningless interval enclosure $[-2.282e+112, 2.282e+112]$ for $y_1(100)$, while `verifyode` stops prematurely because the step size undergoes the minimum step size. With QR preconditioning, COSY and `verifyode` produce very similar enclosures, namely

$$y_1(100) \in [0.14730016186\dots, 0.145593055090\dots].$$

The underlined, trailing digits in the table are those where COSY and `verifyode` differ. Also for test cases 2 and 3 the results obtained with QR preconditioning by COSY and `verifyode` are in good accordance. With shrink wrapping, COSY gives a bit tighter bounds than `verifyode` for test case 2, but for test case 3 COSY computes a wide interval like in case 1 while `verifyode` succeeds with a tight enclosure similar to QR preconditioning. We recall that `verifyode` uses a different kind of shrink wrapping than COSY, cf. [16].

The next example is the Lorenz System

$$\begin{aligned} y_1' &= \sigma(y_2 - y_1) \\ y_2' &= (\rho - y_3)y_1 - y_2 \\ y_3' &= y_1y_2 - \beta y_3 \end{aligned}$$

for the parameters $\sigma := 10$, $\rho := 28$, and $\beta := 8/3$. It is implemented in MATLAB by

```
function dydt = lorenz(t,y,i)
    sigma = 10;
    rho = 28;
    if nargin == 2 || isempty(i) || i == 3
        if isfloat(y)
            beta = 8/3; % only for MATLAB's nonverified solvers like ode45
```

⁸ Thanks to M. Neher [18], Karlsruhe Institute of Technology, for providing more digits of the COSY results than stated in [12].

```

else
    beta = iv2intval(iv_rdivide(8,3)); % verified enclosure of 8/3
end
end
if nargin == 2 || isempty(i)
    dydt = y;
    dydt(1) = sigma.*(y(2)-y(1));
    dydt(2) = (rho-y(3)).*y(1)-y(2);
    dydt(3) = y(1).*y(2)-beta.*y(3);
else
    switch i
    case 1
        dydt = sigma.*(y(2)-y(1));
    case 2
        dydt = (rho-y(3)).*y(1)-y(2);
    case 3
        dydt = y(1).*y(2)-beta.*y(3);
    end
end
end

```

The third input parameter i serves for computing the i th component of the ODE function separately. This is only done for performance reasons since Taylor model arithmetic is expensive and the algorithm sometimes only needs specific components. The system shall be solved for interval initial values

$$y_0 := ([-8.001, -7.998], [7.998, 8.001], [26.998, 27.001])^T,$$

integration period $t \in [0, 3]$, minimum step size $h_{\min} := 0.001$, Taylor model order $m := 10$, and local error tolerance $1e-11$ as done in [8], pp. 150–156. The following program computes two verified solutions with our solver `verifyode`. The first uses QR preconditioning (QR), the second uses shrink wrapping (sw). Furthermore, MATLAB's nonverified solver `ode45` is used to compute an approximate solution of high accuracy where the midpoint of y_0 is taken as initial value. The code shows that calling `verifyode` is very similar to calling `ode45`.

```

t0 = 0; tf = 3;
y0 = intval('[-8.001,-7.998] [7.998,8.001] [26.998,27.001]'); %verified enclosure of y0

% 1. verifyode with QR preconditioning
options = verifyodeset('order',10,'shrinkwrap',0,'precondition',1,'h0',0.05,
    'h_min',1e-3,'loc_err_tol',1e-11,'sparsity_tol',1e-20);
[Tp,Yp,Yrp] = verifyode(@lorenz,[t0,tf],y0,options);

% 2. verifyode with shrink wrapping
options = verifyodeset('order',10,'shrinkwrap',1,'precondition',0,'h0',0.05,
    'h_min',1e-3,'loc_err_tol',1e-11, 'sparsity_tol',1e-20);
[Ts,Ys] = verifyode(@lorenz,[t0,tf],y0,options);

%3. ode45 with high accuracy
options_ode45 = odeset('RelTol',1e-13,'AbsTol',[1e-13 1e-13 1e-13]);
[t_ode45,y_ode45] = ode45(@lorenz,[t0,tf],mid(y0),options_ode45);

```

The function `plottaylormodel_1d` is used to plot enclosures that are overall verified, i.e., zooming in at any position always shows red verified lower and green verified upper bounds of the solution. The right picture below shows such a zoom for y_2 near $t = 3$ (approximate solution by `ode45` in blue) (see Fig. 5).

```

figure; title('Solution of Lorenz system with verifyode'); hold on
plottaylormodel_1d(Ys(:,1),[],2,2,2); % enclosure for y1
plottaylormodel_1d(Ys(:,2),[],2,2,2); % enclosure for y2
plottaylormodel_1d(Ys(:,3),[],2,2,2); % enclosure for y3
t = t_ode45; y = y_ode45;
fig = plot(t,y(:,1),'k',t,y(:,2),'b',t,y(:,3),'c'); % ode45 result
xlabel('Time t'), ylabel('Solution y'), legend(fig,'y_1','y_2','y_3')
axis([ 2.8 3.0 -0.4 0.8]); % Zoom in on the plot for y_2 near t = 3.

```

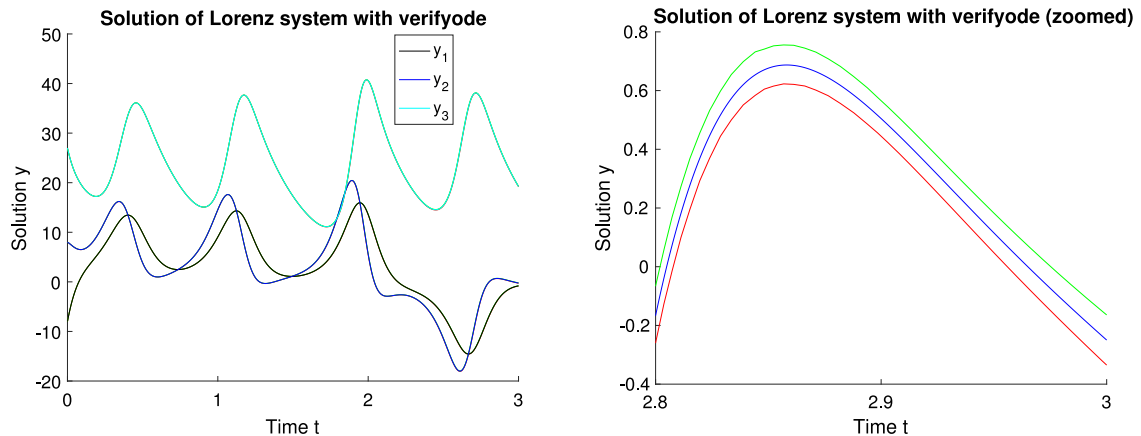


Fig. 5. Lorenz system.

The following table shows enclosures of y_1 , y_2 , and y_3 at the final time $t = 3$ obtained by COSY, Riot, and *verifyode*. Below each enclosure the interval diameter $d([y])$ is listed. The COSY and Riot results are taken from [8], p. 154, Table 5.4. The computing times stated in the last column shall only give an impression. They are not exactly comparable since different computers were used for COSY and Riot on the one hand and *verifyode* on the other, see footnote 7.

	y_1	y_2	y_3	Time [s]
COSY sw	$-7.645\ 588$	$-1.916\ 456$	$1.9^{29\ 640}_{15\ 530}\text{e}+1$	52.54
$d([y])$	$-8.442\ 697$	$-3.023\ 880$	$1.42\text{e}-1$	
RIOT sw	$-7.785\ 478$	$-2.117\ 617$	$1.9^{27\ 413}_{17\ 987}\text{e}+1$	1638.26
$d([y])$	$-8.349\ 144$	$-2.881\ 471$	$9.43\text{e}-2$	
<i>verifyode</i> sw	$-7.830\ 294$	$-2.144\ 718$	$1.9^{26\ 744}_{18\ 616}\text{e}+1$	122.73
$d([y])$	$-8.302\ 504$	$-2.852\ 674$	$8.13\text{e}-2$	
COSY QR	$-7.928\ 041$	$-2.321\ 196$	$1.92^3_{1\ 690}\text{e}+1$	26.73
$d([y])$	$-8.160\ 224$	$-2.620\ 030$	$1.74\text{e}-2$	
<i>verifyode</i> QR	$-7.892\ 306$	$-2.274\ 993$	$1.92^3_{1\ 854}\text{e}+1$	44.97
$d([y])$	$-8.240\ 013$	$-2.722\ 123$	$1.65\text{e}-2$	

With shrink wrapping *verifyode* gives a bit tighter results than COSY and Riot. With QR preconditioning COSY gives slightly better bounds for the first two small components y_1 and y_2 while *verifyode* yields a tighter bound for the large last component y_3 . In general, the results are in reasonable accordance and also the run times of COSY and *verifyode* relate well.

Next, we consider an example given in [19]: the Lotka–Volterra equations

$$y_1' = 2y_1(1 - y_2)$$

$$y_2' = y_2(y_1 - 1)$$

with quite wide interval initial values

$$y_0 := (1, 3)^T \pm 0.05$$

– for which the Taylor model method is especially well-suited – and integration period

$$t \in [0, 5.488138468035]$$

which is approximately a full predator prey cycle. This is a problem that AWA cannot solve. Enclosures for y_1 and y_2 at the end of integration computed by COSY are taken from [8], p. 148, Table 5.3. (Parameter settings like Taylor model order $m := 18$ are also taken from there.) The following table compares these results with those obtained by *verifyode*. COSY gives slightly better bounds, run times are in reasonable accordance although our MATLAB/INTLAB code is interpreted.

	y_1	y_2	Time [s]
COSY QR	1.240 265	3.045 759	7.17
$d([y])$	0.816 719	2.935 927	
	0.424	0.110	
<i>verifyode</i> QR	1.240 296	3.058 082	5.46
$d([y])$	0.780 226	2.932 311	
	0.460	0.126	

A serious example for long term integration is the Kepler problem

$$y'' = \frac{-\gamma y}{(y_1^2 + y_2^2 + y_3^2)^{3/2}}, \quad \gamma := 0.9986,$$

for the near earth asteroid 1997 XF11, see [20], p. 144 et seq., [21], and [8], p. 161 et seq.. Here y_1, y_2, y_3 are the (x, y, z) -coordinates of the asteroid in the solar system with the sun in its center. Rewritten as a six-dimensional ODE this reads

$$y'_i = y_{i+3}, \quad y'_{i+3} = -\gamma \frac{y_i}{d}, \quad i = 1, 2, 3, \quad \text{where } d := (y_1^2 + y_2^2 + y_3^2)^{3/2}.$$

Interval initial conditions come from observations in the year 1997:

$$\begin{aligned} y_1(0) &\in -1.77269098191512 \pm 0.5 \cdot 10^{-7} \\ y_2(0) &\in 0.1487214852342955 \pm 0.5 \cdot 10^{-7} \\ y_3(0) &\in -0.07928350462244194 \pm 0.5 \cdot 10^{-7} \\ y_4(0) &\in 0.2372031916516237 \pm 0.5 \cdot 10^{-6} \\ y_5(0) &\in 0.612524538758628 \pm 0.5 \cdot 10^{-6} \\ y_6(0) &\in 0.04583217572165624 \pm 0.5 \cdot 10^{-6} \end{aligned}$$

Long term integration is done for 23 years from 1997 to 2020. In scaled time units one year corresponds to 2π so that the integration period is $[t_0, t_f] = [0, 46\pi]$.

In [8], p. 163, Table 5.6, results for COSY executed with a method called QR/blunting (QR-BL) are stated.⁹ In contrast, we use blunted parallelepiped preconditioning (PE-BL) where the matrix $Q := A$ in (12) is replaced by a blunted matrix $\tilde{Q}$ having a possibly better condition number. Without going into details, our newly implemented blunting method takes columnwise different blunting factors in a similar way as described in [17].¹⁰ Furthermore, we choose a more sensitive relative inflation constant $\varepsilon := 0.01$ (1 percent) in (11) instead of the default value $\varepsilon := 0.1$ (10 percent). Both, COSY and `verifyode`, use Taylor model order $m := 10$. We mention that AWA breaks down after nearly 8 years ($t_f = 16\pi$) and also Riot does not succeed. The following table compares the results of COSY and `verifyode`. The inclusion intervals computed by `verifyode` are slightly tighter than those of COSY but the diameters, stated in the columns $d([y])$, are of the same order.

	COSY (QR-BL)		verifyode (PE-BL)	
		$d([y])$		$d([y])$
y_1	$-9.0^{29 \ 650}_{39513} e-1$	$9.87e-4$	$-9.03^{1 \ 668}_{7 \ 466} e-1$	$5.80e-4$
y_2	$-7.49^{4 \ 944}_{9 \ 257} e-1$	$4.32e-4$	$-7.49^{5 \ 868}_{8 \ 346} e-1$	$2.48e-4$
y_3	$8.6^{91 \ 249}_{24 \ 310} e-3$	$6.70e-5$	$8.6^{76 \ 153}_{39 \ 600} e-3$	$3.66e-5$
y_4	$9.2^{33 \ 148}_{27 \ 239} e-1$	$5.91e-4$	$9.2^{31 \ 951}_{28 \ 454} e-1$	$3.50e-4$
y_5	$-3.9^{66 \ 078}_{71 \ 035} e-1$	$4.96e-4$	$-3.9^{67 \ 097}_{70 \ 004} e-1$	$2.91e-4$
y_6	$6.02^{7 \ 035}_{6 \ 357} e-2$	$6.78e-6$	$6.026^{910}_{482} e-2$	$4.28e-6$
Time [s]	$16 \ 316 \hat{=} 4.53 \text{ h}$		$14 \ 509 \hat{=} 4.04 \text{ h}$	

In analogy to [21], Figure 5, the following picture shows the evolution of the diameters of y_1, y_2, y_3 . Starting from a common diameter 10^{-7} in the initial conditions the diameters increase during integration by less than 4 orders which seems reasonable for such a long integration period (see Fig. 6).

Finally we consider the double pendulum.¹¹ Denoting the pendulum lengths and masses by ℓ_i and m_i , $i = 1, 2$, the corresponding second order ODE reads

$$\begin{pmatrix} \psi_1'' \\ \psi_2'' \end{pmatrix} = \begin{pmatrix} (m_1 + m_2)\ell_1 & m_2\ell_2 \cos(\psi_2) \\ m_2\ell_1 \cos(\psi_2) & m_2\ell_2 \end{pmatrix}^{-1} \begin{pmatrix} -g(m_1 + m_2) \sin(\psi_1) + m_2\ell_2 \sin(\psi_2)(\psi_1' + \psi_2')^2 \\ -gm_2 \sin(\psi_1 + \psi_2) - m_2\ell_1 \sin(\psi_2)\psi_1'^2 \end{pmatrix},$$

where we set $g := 9.81$. For $m_1 = m_2 = 1$ and $\ell_1 = \ell_2 = 1$ this becomes

$$\begin{pmatrix} \psi_1'' \\ \psi_2'' \end{pmatrix} = \frac{1}{1 + \sin^2(\psi_2)} \begin{pmatrix} \cos(\psi_2)(g \sin(\psi_1 + \psi_2) + \sin(\psi_2)\psi_1'^2) - 2g \sin(\psi_1) + \sin(\psi_2)(\psi_1' + \psi_2')^2 \\ \cos(\psi_2)(2g \sin(\psi_1) - \sin(\psi_2)(\psi_1' + \psi_2')^2) - 2(g \sin(\psi_1 + \psi_2) + \sin(\psi_2)\psi_1'^2) \end{pmatrix}.$$

⁹ From the description in [8] it is not clear to us how blunting is combined with QR preconditioning in this method.

¹⁰ We want to notice clearly that this kind of blunting is not contained in the actual INTLAB release V11 but will be part of the next release V11.1 which is planned to be available in the end of 2019/beginning of 2020. We plan to include this example in the INTLAB documentation of the Taylor model toolbox as well so that it can be recomputed.

¹¹ Thanks to one of the reviewers for suggesting this example.

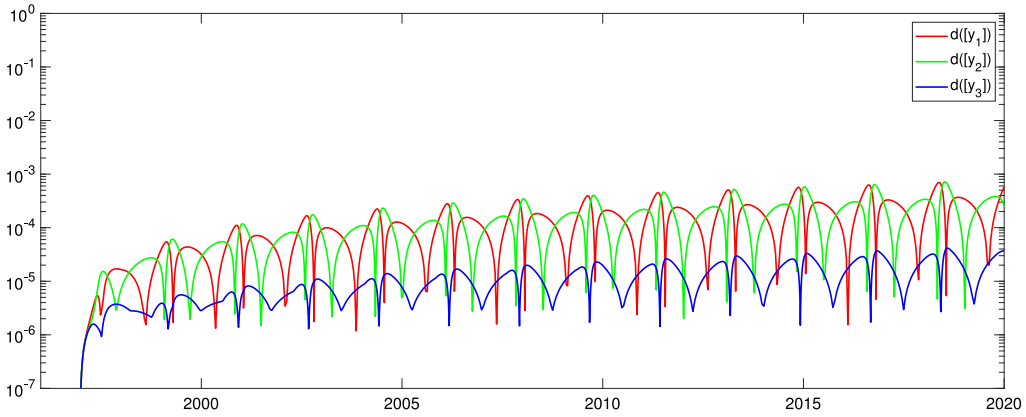


Fig. 6. Diameter evolution.

An efficient MATLAB/INTLAB implementation of the corresponding four-dimensional first order ODE function with $y_i := \psi_i$, $y_{i+2} := y'_i$, $i = 1, 2$, may look as follows:

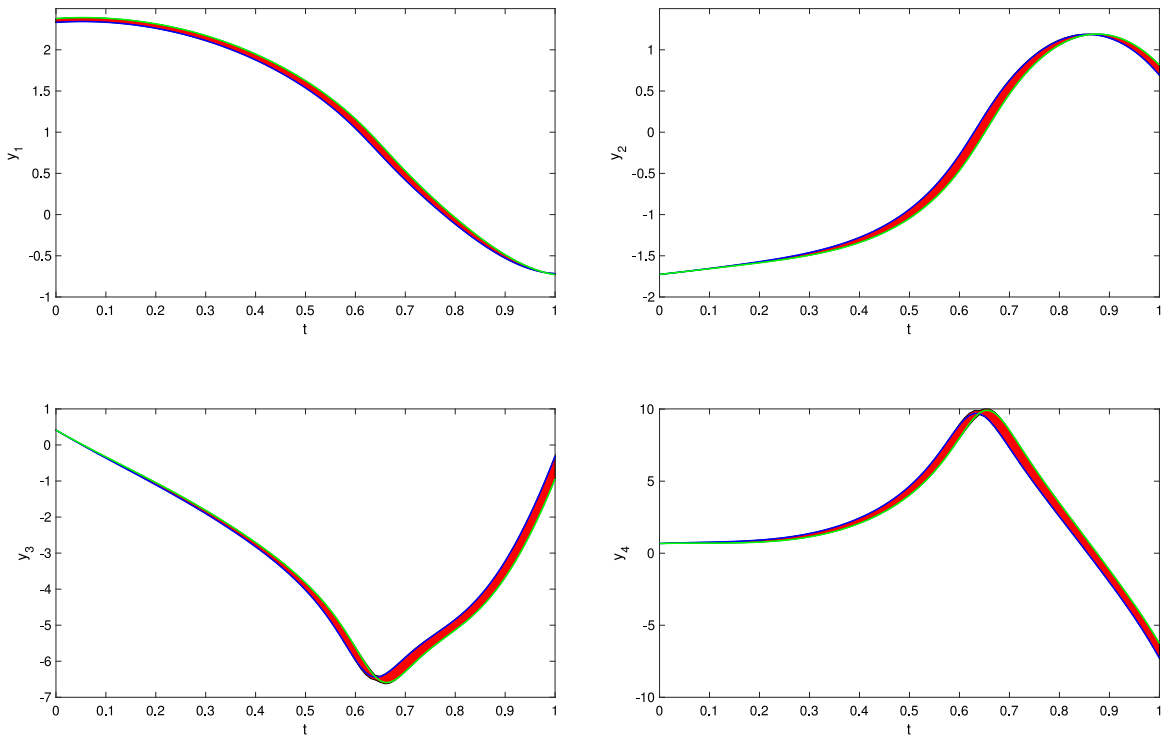
```
function dy = double_pendulum(t,y,i)
persistent g_
if nargin == 2 || isempty(i) || i >= 3
    if isempty(g_)
        g_ = intval('9.81'); % For performance reasons g_ is defined as persistent so
                             % that INTLAB's rigorous enclosure of 9.81 is only computed once.
    end
    if isfloat(y)
        g = 9.81; % only needed for using MATLAB's nonverified solvers like ode45
    else
        g = g_;
    end
    c_01 = sin(y(1)); c_02 = sin(y(2)); c_03 = sqrt(c_02); c_04 = sin(y(1)+y(2));
    c_05 = cos(y(2)); c_06 = sqrt(y(3)+y(4)); c_07 = sqrt(y(3)); c_08 = 1+c_03;
    c_09 = c_02*c_06-2*g*c_01; c_10 = -(g*c_04+c_02*c_07); c_11 = c_09-c_05*c_10;
    if nargin == 2 || (~isempty(i) && i == 4)
        c_12 = 2*c_10-c_05*c_09;
    end
end
if nargin == 2 || isempty(i)
    dy = y;
    dy(1) = y(3);
    dy(2) = y(4);
    dy(3) = c_11/c_08;
    dy(4) = (c_12-c_11)/c_08;
else
    switch i
    case 1
        dy = y(3);
    case 2
        dy = y(4);
    case 3
        dy = c_11/c_08;
    case 4
        dy = (c_12-c_11)/c_08;
    end
end
```

In [22] the following interval initial values were considered:

$$\psi_1(0) \in \frac{3}{4}\pi \cdot [0.99, 1.01], \quad \psi_2(0) := -\frac{11}{20}\pi \approx -1.7279, \quad \psi'_1(0) := 0.43, \quad \psi'_2(0) := 0.67$$

In [10], which relates to [22], slightly different values are taken:

$$\psi_1(0) \in \frac{3}{4}\pi \cdot [0.99, 1.01], \quad \psi_2(0) := -1.726533538, \quad \psi'_1(0) := 0.4138843714, \quad \psi'_2(0) := 0.6724072960 \quad (13)$$

Fig. 7. Double pendulum $t_f = 1$.

In the computations, inclusions of the latter values are used as starting values. Both, [22] and [10], compare the verified ODE solvers VNODE [2], COSY, and VALENCIA [22]. They concentrate on breakdown times; enclosures are not explicitly stated. It is reported that VNODE and VALENCIA breakdown at $t_f < 0.6$ and only COSY could integrate up to $t_f = 1$.¹² Interestingly, wrapping seems not to be the major problem for integrating up to $t_f = 1$. Actually, wrapping diminishing techniques like shrink wrapping or preconditioning have counterproductive effects for the comparatively wide interval initial conditions (13). It turns out that naive Taylor model arithmetic gives best results with best performance. Like in [22] and [10], we choose Taylor model order $m := 12$. Precisely, we execute `verifyode` with the following options:

```
options = verifyodeset('order',12,'shrinkwrap',0,'precondition',0,'blunting',0, ...
                      'h0',0.05,'h_min',1e-2,'loc_err_tol',1e-9,'sparsity_tol',1e-12);
```

and get the following enclosures and run times for $t_f = 0.5$ and $t_f = 1$:

	$t_f = 0.5$	$d([y])$	$t_f = 1$	$d([y])$
y_1	[1.5381, 1.6247]	8.66e-02	[-7.3041, -7.1564]e-1	1.48e-2
y_2	[-1.0452, -9.3597e-1]	1.10e-01	[6.9041, 8.1538e-1]e-1	1.25e-1
y_3	[-4.0246, -3.8218]	2.03e-01	[-9.4656, -3.0284]e-1	6.44e-1
y_4	[4.0274, 4.6513]	6.24e-01	[-7.3001, -6.2542]	1.05
Time [s]	17.0		69.6	

The enclosure $[-7.3001, -6.2542]$ of y_4 at $t_f = 1$ appears to be wide. However, enclosures obtained from computations with point initial data from the initial value set (13) show that this is not primarily caused by overestimation, but by diverging trajectories of distinct solutions of the ODE. To show this, we execute `verifyode` another two times for $t_f = 1$ with point initial values $\psi_1(0) := \frac{3}{4}\pi \cdot 0.99$ and $\psi_1(0) := \frac{3}{4}\pi \cdot 1.1$, and call the results for these two trajectories u and v , respectively. Then, at $t = t_f = 1$ we have $u_4 = [-7.2788, -7.2786]$ and $v_4 = [-6.2646, -6.2644]$ which gives the inner approximation $[-7.2786, -6.2646]$ of y_4 . Thus, the outer and inner approximations of y_4 differ by about three percent in diameter only. Fig. 7 contains the four components of y (red tube), u (blue line), and v (green line). The diameters of u and v are of order 10^{-5} and thus below plotting resolution.

¹² Using a special consistency test for subboxes, VALENCIA was able integrate up to $t_f = 0.9915$.

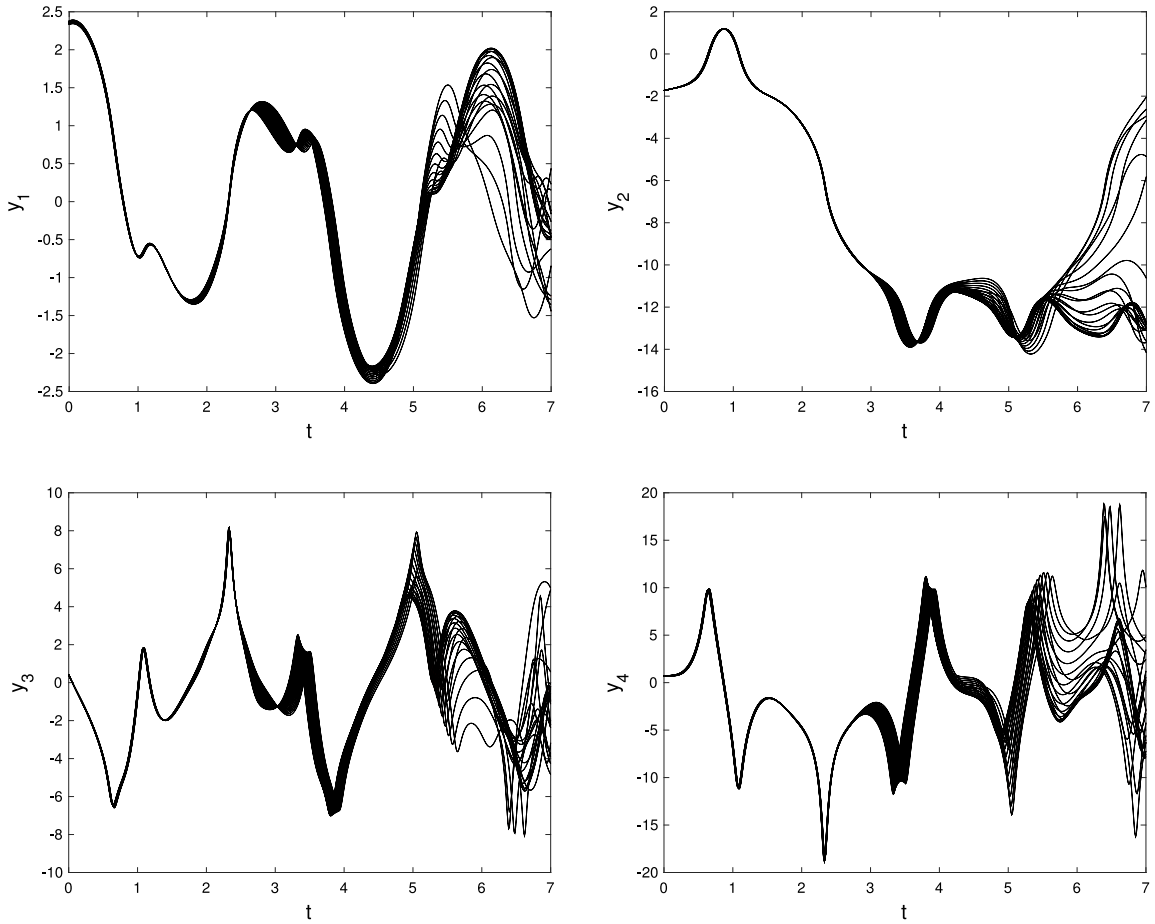


Fig. 8. Double pendulum trajectories $t_f = 7$.

In contrast to short term integration up to $t_f = 1$, longer integration up to, say, $t_f = 7$ is not possible with naive Taylor model arithmetic, even not for point initial values as used for computing u and v . Such point initial values can successfully be integrated with QR-preconditioning, however also other verified solvers like AWA, for example, can easily do that but much faster. The following figure shows the results of `verifyode` for 20 point initial values where $\psi_1(0)$ ranges in $\frac{3}{4}\pi \cdot [0.99, 1.01]$. This corresponds to Figures 1 and 2 of [10]. To be precise, `verifyode` is now executed with:

```
options = verifyodeset('order',12,'shrinkwrap',0,'precondition',1,'blunting',0,...
    'h0',0.5,'h_min',1e-3,'loc_err_tol',1e-11,'sparsity_tol',1e-16);
```

The diameters of the obtained enclosures at $t_f = 7.0$ are of order less than 10^{-4} and thus below plotting resolution (see Fig. 8).

The main purpose of these computations is to show that the wide spread of the single solutions at $t_f = 7$ supplies a wide inner enclosure for the solution of the original interval conditions (13). This clearly shows that it is impossible to perform long term integration for these conditions without using some kind of subdivision strategies for downstream initial value sets.¹³ Having recognized this, it may still be of interest to see how far the tight midpoint enclosure given by

$$y_0 = [3 \cdot \text{intval}(\text{'pi'})/4; \text{intval}(\text{'-1.726533538 0.4138843714 0.6724072960'})] \quad (14)$$

can be integrated without such subdivision strategies. For that we increase the Taylor model order to $m := 16$:

```
options = verifyodeset('order',16,'shrinkwrap',0,'precondition',1,'blunting',0,...
    'h0',0.5,'h_min',1e-3,'loc_err_tol',1e-11,'sparsity_tol',1e-16);
```

¹³ COSY offers automatic domain decomposition for that. Such a feature is not implemented for `verifyode`.

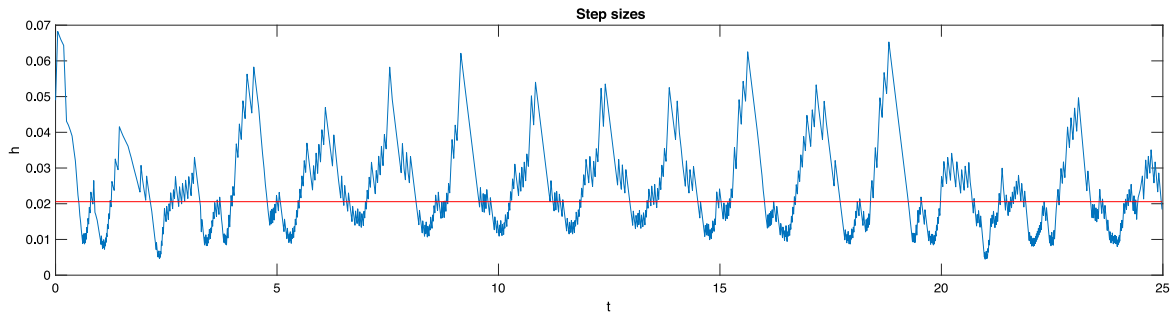


Fig. 9. Step sizes $t_f = 25$.

Moreover, like for the Kepler problem, the relative inflation constant $\varepsilon := 0.01$, see (11), is used. With these settings the maximum integration time is about $t_f = 25$ where the following enclosures are obtained:

	$t_f = 25$	$d([y])$
y_1	[1.7729, 1.8284]	5.54e-02
y_2	[-2.6238, -2.6033]e+1	2.05e-01
y_3	[-2.3541, -2.2659]	8.82e-02
y_4	[4.4325, 6.9119]e-1	2.48e-01
Time [h]	14.5	
Steps	1215	

The long run time is not due too extremely small step sizes – the average step size is about 0.02, see Fig. 9 – but due to the comparatively high complexity of the double pendulum's ODE function which involves multiple sine and cosine evaluations. This points to the weakness of Taylor models: their expensive arithmetic. Here, as stated before, Taylor model multiplication, which is roughly nothing else but discrete convolution, is the main performance bottleneck. It occurs in the implementation of all standard functions.

In the example of the double pendulum the polynomials have five variables (four space-like variables plus time t), degree at most 16, and coefficients of modulus less than $1e-16$ are suppressed. Decreasing the latter two values is unfavorable. Towards the end of the integration domain, around $t = 23$, the polynomials become dense and drastically slow down arithmetical operations. However, the computed enclosures at $t_f = 25$ seem reasonable for the long integration period of this sensitive system. In comparison, AWA is extremely fast. Integrating up to about $t_f = 23$ takes less than 30 s with inclusions of similar accuracy. On the other hand, it breaks down at about $t_f = 23.6$ with wide enclosures of diameter $1e+12$.

Finally, we look at MATLAB's nonverified ODE solver ode45 and execute it three times with default, medium, and high accuracy options $\text{RelTol}/\text{AbsTol} = 1e-3/1e-6, 1e-6/1e-9, 1e-14/1e-14$, respectively:

```

odefun = @double_pendulum;
tspan = [0 25];
y0 = [3*intval('pi')/4;intval('-1.726533538 0.4138843714 0.6724072960')];
options_2 = odeset('RelTol',1e-6,'AbsTol',[1e-9 1e-9 1e-9 1e-9]); % medium accuracy
options_3 = odeset('RelTol',1e-14,'AbsTol',[1e-14 1e-14 1e-14 1e-14]); % high accuracy
[Ts_1,Ys_1] = ode45(odefun,tspan,mid(y0)); % default accuracy RelTol = 1e-3, AbsTol = 1e-6
[Ts_2,Ys_2] = ode45(odefun,tspan,mid(y0),options_2); % medium accuracy
[Ts_3,Ys_3] = ode45(odefun,tspan,mid(y0),options_3); % high accuracy
load result_verifyode.mat T Y Yr; % The previously computed verifyode result is read from file.
n = length(y0); idx = n+1; mode = 1; kmax1 = 1; kmax2 = 3; kmax3 = 3;
for i = 1:n
    figure(i), hold on
    plot(Ts_1,Ys_1(:,i),'--b',Ts_2,Ys_2(:,i),'-.m',Ts_3,Ys_3(:,i),'k') % Plot ode45 results.
    plottaylormodel_id(Y(:,i),idx,kmax1,kmax2,kmax3,mode); % Plot verifyode result.
    legend('ode45 : default accuracy','ode45 : medium accuracy','ode45 : high accuracy','y.inf','y.sup')
    xlabel('t');
    ylabel(['y_',num2str(i)]);
    title(['Component y_',num2str(i),'(t)']);
end

```

The plots for the first two components $y_i(t) = \psi_i(t)$, $i = 1, 2$, are given below (see Fig. 10). Default accuracy (blue dashed line) gives obvious wrong results starting from $t \approx 5$, and medium accuracy (magenta dash-dotted line) from $t \approx 19$. Conversely, zooming in at $t = 25$ shows that the high accuracy floating-point approximation (black solid line) is enclosed by the result computed with verifyode (red and green solid lines for lower and upper bounds) (see Fig. 11).

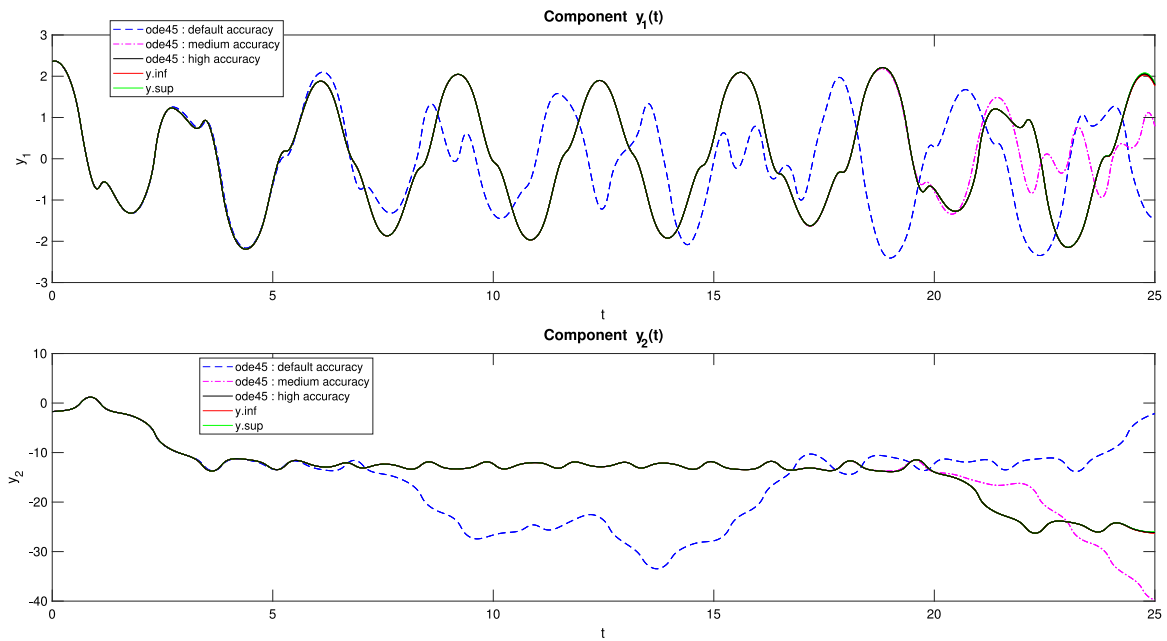


Fig. 10. verifyode vs. ode45.

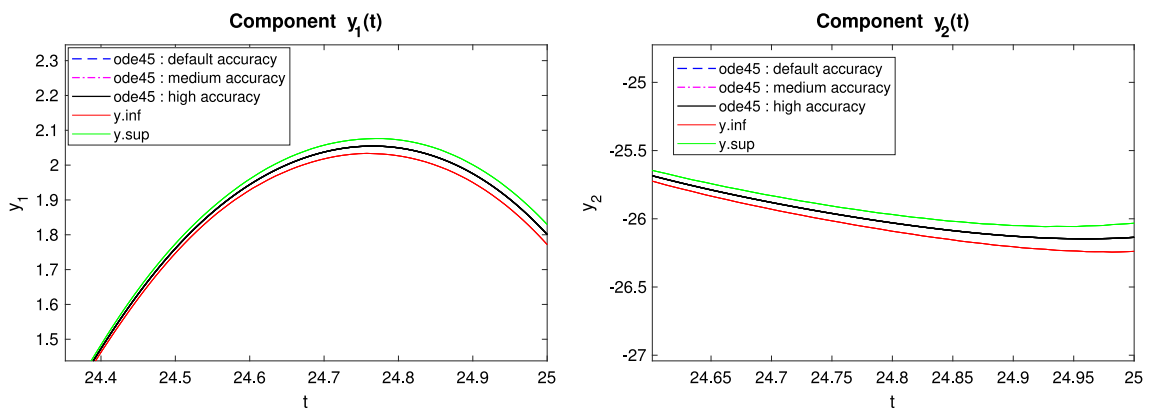


Fig. 11. verifyode vs. ode45 zoomed.

Acknowledgments

I thank S.M. Rump for many valuable comments and suggestions. Furthermore, I thank the two unknown reviewers for their helpful remarks and suggestions.

References

- [1] R. Lohner, Einschließung der Lösung Gewöhnlicher Anfangs- und Randwertaufgaben und Anwendungen, Dissertation at Karlsruhe Institute of Technology, 1988, Software: AWA, http://www2math.uni-wuppertal.de/xsc/xsc/pxsc_software.html#awa.
- [2] N.S. Nedialkov, VNODE-LP - a validated solver for initial value problems in ordinary differential equations. Technical Report CAS-06-06-NN, Department of Computing and Software, McMaster University, Hamilton, Canada, L8S4K1, 2006, Software: VNODE-LP, <http://www.cas.mcmaster.ca/nedialk/vnodelp/>, VNODE, <http://www.cas.mcmaster.ca/%7Enedialk/Software/VNODE/VNODE.shtml>.
- [3] M. Berz, et al., The COSY INFINITY web page, www.bt.pa.msu.edu/index_cosy.htm.
- [4] M. Kashiwagi, kv - a C++ library for verified numerical computation, <http://verifiedby.me/kv/index-e.html>.
- [5] Y. Lin, M.A. Stadtherr, Validated solution of initial value problems for ODEs with interval parameters, in: R. L. Muhanna, R. L. Mullen (Eds.), Proceedings of 2nd NSF Workshop on Reliable Engineering Computing, Savannah, GA, 2006.
- [6] X. Chen, Reachability analysis of non-linear hybrid systems using Taylor models, Dissertation at RWTH Aachen University, 2015, Software: FLOW, <https://flowstar.org/downloads/>.

- [7] T. Dzetkulič, Rigorous integration of non-linear ordinary differential equations in Chebyshev basis, *Numer. Algorithms* 69 (2015) 183–205, Software: ODEintegrator, <https://sourceforge.net/projects/odeintegrator/>.
- [8] I. Eble, Über Taylor-Modelle, Dissertation at Karlsruhe Institute of Technology, 2007, Software: RIOT, www.math.kit.edu/ianm1/ingo.eble/de.
- [9] S.M. Rump, INTLAB-INTERVAL LABORATORY, in: Tibor Csentes (Ed.), *Developments in Reliable Computing*, Kluwer Academic Publishers, Dordrecht, 1999, pp. 77–104.
- [10] M. Berz, K. Makino, Rigorous integration of flows and ODEs using Taylor models, in: *Proceedings of the 2009 conference on symbolic numeric computation*, 2009, pp. 79–84, <http://dx.doi.org/10.1145/1577190.1577206>.
- [11] S.M. Rump, *Kleine Fehlerschranken bei Matrixproblemen*, Dissertation at Karlsruhe University, 1980.
- [12] M. Neher, K.R. Jackson, N.S. Nedialkov, On Taylor model based integration of ODEs, *SIAM J. Numer. Anal.* 45 (1) (2007) 236–262.
- [13] M. Berz, K. Makino, Suppression of the wrapping effect by Taylor model-based verified integrators: long-term stabilization by shrink wrapping, *Int. J. Differential Equations Applications* 10 (4) (2005) 385–403.
- [14] M. Berz, K. Makino, Suppression of the wrapping effect by Taylor model-based verified integrators: long-term stabilization by preconditioning, *Int. J. Differential Equations Applications* 10 (4) (2005) 105–136.
- [15] K. Makino, Taylor model-based verified integrators, in: *Seventh International Workshop on Taylor Methods*, Key West, Florida, 2011, Slides: <https://www.bt.pa.msu.edu/TM11/talks/makino.pdf>.
- [16] F. Bünger, Shrink wrapping for Taylor models revisited, *Numer. Algorithms* 78 (4) (2018) 1001–1017.
- [17] M. Neher, K.R. Jackson, N.S. Nedialkov, On the blunting method in verified integration of ODEs, *Reliab. Comput.* 23 (2016) 15–34.
- [18] M. Neher, Personal communication, 2017.
- [19] M. Berz, K. Makino, Suppression of the wrapping effect by Taylor model-based verified integrators: the single step, *Int. J. Pure Appl. Math.* 36 (2) (2006) 175–197.
- [20] J. Hoefkens, *Rigorous numerical analysis with high-order Taylor models*, Dissertation at Michigan State University, 2001.
- [21] J. Hoefkens, M. Berz, K. Makino, Controlling the wrapping effect in the solution of ODEs for asteroids, *Reliab. Comput.* 8 (2003) 21–41.
- [22] A. Rauh, E.P. Hofer, E. Auer, VALENCIA-IVP: A comparison with other initial value problem solvers, Published in: 12th GAMM - IMACS International Symposium on Scientific Computing, Computer Arithmetic and Validated Numerics, SCAN 2006.